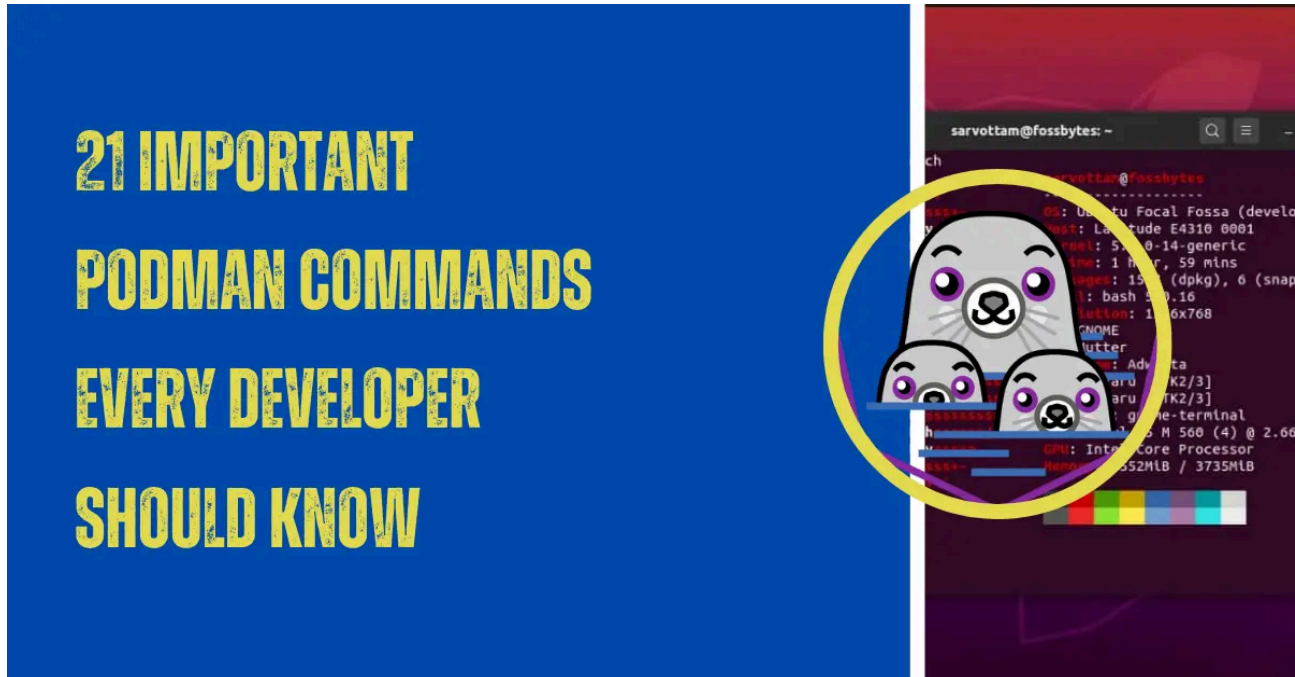


# 21 Important Podman Commands Every Developer Should Know (2026 Updated)

 [zihad.com.bd/posts/podman-commands](https://zihad.com.bd/posts/podman-commands)

November 3, 2025



Learn the top 21 Podman commands every developer should know in 2026. From basic container management to advanced pod usage — boost your Linux container skills fast with this cheat sheet!

## What is Podman?

If you love Docker but want a more secure and rootless alternative, Podman is for you. Podman (short for Pod Manager) is a daemonless container engine developed by Red Hat. It lets you run containers without root access, and it's fully compatible with Docker commands.

Think of Podman as Docker without the daemon — lightweight, secure, and perfect for developers, sysadmins, and DevOps engineers.

## 1. Check Podman Version

```
podman
version
```

Displays podman version informations. Output:

```
$ podman version
Client:      Podman Engine
Version:     5.6.2
API Version: 5.6.2
Go Version:  go1.22.5
Built:       2025-08-
17T14:32:12Z
OS/Arch:     linux/amd64
```

## 2. Display System Information

---

```
podman
info
```

Shows detailed info about storage drivers, registries, and system configuration.

## 3. Search for image on registry

---

```
podman search
httpd
```

This searches for httpd in registry. Output:

```
> podman search httpd
```

| NAME                                 | DESCRIPTION                               |
|--------------------------------------|-------------------------------------------|
| registry.fedoraproject.org/f29/httpd |                                           |
| docker.io/library/httpd              | The Apache HTTP Server Project            |
| docker.io/manageiq/httpd             | Container with httpd, built on CentOS for |
| Ma...                                |                                           |

## 4. Pull the Image

---

```
podman pull
registry.access.redhat.com/ubi9/httpd-24
```

This pull the image from redhat registry

## 5. List all images in local system

---

```
podman
images
```

It will list podman images Output:

```
$ podman images
REPOSITORY                                TAG          IMAGE ID      CREATED
SIZE
docker.io/library/ubuntu                  latest       62ff5a5d9d4d  2 weeks ago
78.3 MB
docker.io/library/nginx                   latest       7b36f48b5c7e  3 weeks ago   187
MB
quay.io/podman/hello                      latest       a23f9470bc18  4 weeks ago
12.4 MB
docker.io/library/mysql                   8.0          79feff3f8b32  1 month ago   564
MB
registry.fedoraproject.org/fedora         41           21e6e8ccf3f7  2 months ago  182
MB
```

## 6. Run container

---

```
podman run -d --name myweb
registry.access.redhat.com/ubi9/httpd-24
```

- -d means detached mode, it will run in background.
- --name is meaningful container name.

## 7. List running containers

---

```
podman
ps
```

It will show running containers.

```
$ podman ps
CONTAINER ID  IMAGE                                COMMAND                                CREATED
STATUS        PORTS  NAMES
c7b9e4a2d78b  registry.access.redhat.com/ubi9/httpd-24  /usr/sbin/httpd -D...  5 seconds
ago         Up 5 seconds ago                  myweb
```

```
podman ps
-a
```

This will show all commands even if it's stopped.

## 8. Stop the container

---

```
podman stop
myweb
```

Podman stop will stop container gracefully.

## 9. Delete & remove the container

---

```
podman container rm -f
myweb
```

This will forcefully remove the container even if it's running.

## 10. Delete the image from system

---

```
podman image rm
registry.access.redhat.com/ubi9/httpd-24
```

This will remove the image from system and clean up your environment.

## 11. podman log

---

Let's start a database container.

```
podman run -d --name mydb
docker.io/library/mariadb
```

You will notice this podman didn't run using `podman ps`

You can check log using

```
podman logs
mydb
```

## 12. Podman inspect

---

We use inspect to check relevant info to run the container

```
podman image inspect registry.fedoraproject.org/f31/mariadb | jq -r '.[0].Labels.usage'
```

## 13. Environment variables

---

Let's rerun the container passing the env variables

```
podman run -d -e MYSQL_USER=myuser -e MYSQL_PASSWORD=mypass -e MYSQL_DATABASE=mydb --name mydb registry.fedoraproject.org/f31/mariadb
```

-e environmental variables

## 14. Networking

---

Let's pull an image of apache webserver.

```
podman pull  
registry.fedoraproject.org/f29/httpd
```

Let's pull curl image

```
podman pull  
docker.io/curlimages/curl
```

Start the apache server

```
podman run -dit --name myserver -p 8080:8080  
registry.fedoraproject.org/f29/httpd
```

Start the curl server. We will try to access apache server through curl image.

```
podman run --name myclient docker.io/curlimages/curl  
myserver:8080
```

It should fail.

```
curl: (6) Could not resolve host:
myserver
```

Because it can't access dns. Let's check what network is available for our images.

```
podman
network ls
```

Only podman image is available. We can inspect it.

```
podman network inspect
podman
```

We can see that "dns\_enabled": false,. That's why curl can't access apache server.

Let's create our own network with DNS enabled.

```
podman network create mynetwork --subnet 192.168.1.0/24 --gateway
192.168.1.254
```

We can't inspect that DNS is enabled.

```
podman network inspect
mynetwork
```

Let's remove previous containers. Restart containers.

```
podman container rm -f
myserver
podman container rm -f
myclient
```

Run apache server using our newly created network

```
podman run -dit --name myserver --network mynetwork -p 8080:8080
registry.fedoraproject.org/f29/httpd
```

Now let's try to connect our DNS enabled apache server using curl container.

```
podman run --name myclient --network mynetwork docker.io/curlimages/curl myserver:8080 |  
head
```

Let's check without our system curl if it can connect with our container

```
curl localhost:8080 |  
head
```

It fails. This confirms our setup is correct. Outside network can't access our server.

## 15. Volume management

---

Create a local directory to use as a bind mount for serving web content.

```
mkdir  
web_content
```

Add a sample HTML file inside your bind mount directory.

```
echo 'hello from bind mount' >  
web_content/index.html
```

Run an Apache container using the bind mount, forwarding host port 8080 to container port 80.

```
podman run -d --name httpd_bind -v ./web_content:/usr/local/apache2/htdocs:Z -p 8080:80  
docker.io/library/httpd
```

Test that your Apache server is serving the bind-mounted content on port 8080.

```
curl  
localhost:8080
```

Create a new named Podman volume for persistent storage.

```
podman volume create  
my_volume
```

List all volumes on your system to verify the newly created volume exists.

```
podman  
volume ls
```

Display detailed information about the named volume, including its mount path.

```
podman volume inspect  
my_volume
```

Run an Apache container using the named volume, exposing it on host port 8090.

```
podman run -d --name httpd_volume -v my_volume:/usr/local/apache2/htdocs:Z -p 8090:80  
docker.io/library/httpd
```

write inside volume

```
podman exec -t httpd_volume sh -c 'echo "hello from volume mount" >  
/usr/local/apache2/htdocs/index.html'
```

Verify that the container is serving the volume-mounted content on port 8090.

```
curl  
localhost:8090
```

Export the content of the named volume to a compressed tarball for backup.

```
podman volume export my_volume --output  
my_volume.tar.xz
```

Create a new volume to import your backup into.

```
podman volume create my_new_volume  
podman volume import my_new_volume  
my_volume.tar.xz
```

Run a new Apache container using the restored volume, accessible on host port 9090.

```
podman run -d --name httpd_volume -v my_new_volume:/usr/local/apache2/htdocs:Z -p 9090:80  
docker.io/library/httpd
```

test restored volume

```
curl
localhost:9090
```

## 16. Podman secrets

---

Let's create our secret files that contains secret data such as username or passwords.

```
mkdir secrets && cd secrets
echo -n 'my_user_name' >
DB_user.txt
echo -n 'myDB' > DB_name.txt
echo -n 'redhat' > DB_pass.txt
echo -n 'redhat' >
DB_root_pass.txt
```

Let's create our podman secrets using this files.

```
podman secret create DB_user DB_user.txt
podman secret create DB_pass DB_pass.txt
podman secret create DB_name DB_name.txt
podman secret create DB_root_pass
DB_root_pass.txt
```

We can check and inspect our secrets

```
podman secret ls
podman secret inspect
DB_name
```

Let's use our secrets in a container

```
podman run -d \
  --name myDB \
  --secret
DB_root_pass,type=env,target=MYSQL_ROOT_PASSWORD \
  --secret DB_name,type=env,target=MYSQL_DATABASE \
  --secret DB_user,type=env,target=MYSQL_USER \
  --secret DB_pass,type=env,target=MYSQL_PASSWORD \
  -p 3306:3306 \
  docker.io/library/mariadb:latest
```

We can check if our secrets working as intended

```
podman exec -it myDB  
env
```

## 17. Clean up unused containers

---

```
podman system  
prune -a
```

## 18. podman system usage

---

Shows each container using system resources

```
podman  
stats
```

## 19. System Maintenance & Cleanup

---

Display disk space used by images, containers, and volumes.

```
podman  
system df
```

## 20. Copy a file from inside a container to your host.

---

You can backup any container data to host

```
podman cp myweb:/usr/local/apache2/htdocs/index.html  
./backup.html
```

## 21. Reset Podman

---

Make sure you backup everything. This will delete everything. And Reset podman to a clean state.

```
podman system  
reset
```